# Análisis de desempeño y optimización usando paralelización para una solución de la ecuación de Poisson 2D

## Taller OpenMP

Diego A. Acosta V. Oscar J. Cogua A.

Facultad de Ciencias Matemáticas y Naturales
Sistemas Distribuidos — Universidad Distrital Francisco José de Caldas

25 de marzo de 2026

# Contenido


1. Introducción y Marco Teórico
2. Versión Secuencial
3. Actividades OpenMP
4. Comparativa de Resultados
5. Guía de Directivas OpenMP
6. Tabla Comparativa entre Métodos
7. Conclusiones

# Problema: Ecuación de Poisson 2D

## Ecuación diferencial

$$\nabla^2 V(x,y) = 4 \quad \Omega = [1,2] \times [0,2]$$

## Condiciones de frontera (Dirichlet)

$$V(1,y) = (1-y)^2 \qquad V(2,y) = (2-y)^2$$
$$V(x,0) = x^2 \qquad V(x,2) = (x-2)^2$$

## Solución analítica exacta

$$V(x,y) = (x-y)^2$$

## Esquema numérico — 5 puntos

$$V_{i,j}^{\mathbf{new}} = \frac{(V_{i\pm 1,j})k^2 + (V_{i,j\pm 1})h^2 - 4h^2k^2}{2(h^2+k^2)}$$

## Parámetros experimentales

- Malla: $1024 \times 1024$
- $h = 1/M, \quad k = 2/N$
- $\mathrm{TOL} = 10^{-6}$
- 20 hilos (`OMP_NUM_THREADS=20`)
- Iteración: **Gauss-Seidel in-place**

## Nota sobre paralelización

Al paralelizar, el esquema se comporta como **Jacobi implícito**, lo que puede alterar el número de iteraciones.

# Versión Secuencial (Base)

```
double delta = 1.0;
iterations = 0;
while (delta > TOL) {
    delta = 0.0;
    for (int i = 1; i < M; ++i)
        for (int j = 1; j < N; ++j) {
            double V_new = /* esquema 5 pts */;
            delta = std::max(delta,
                        std::abs(V_new-V[i][j]));
            V[i][j] = V_new;
        }
    ++iterations;
}
```

## Resultados

| Métrica | Valor |
|---|---|
| Tiempo | 1 657.98 s |
| Iteraciones | 236 161 |
| Hilos | 1 |
| Speedup | 1,0× |

## Línea base

Esta versión es la **referencia** para calcular el speedup de todas las estrategias paralelas.

# Actividad 1: parallel for + reduction

```
#pragma omp parallel for reduction(max:delta)
for (int i = 1; i < M; ++i)
    for (int j = 1; j < N; ++j) {
        double V_new = /* esquema 5 pts */;
        delta = std::max(delta,
                    std::abs(V_new-V[i][j]));
        V[i][j] = V_new;
    }
```

### ¿Por qué reduction(max:delta)?

Sin reduction: múltiples hilos escriben delta simultáneamente → **condición de carrera**. La cláusula crea una copia privada por hilo y combina con max() al finalizar.

### ¿Cambió el número de iteraciones?

Mínimamente: 236 164 vs 236 161 (+3). El orden de actualización cambia al distribuir filas entre hilos.

## Resultados

| Métrica | Valor |
|---|---|
| Tiempo | 102.39 s |
| Iteraciones | 236 164 |
| Speedup | 16,2× |
| Eficiencia | ≈ 81 % |

### Mejor resultado global

**Mejor relación rendimiento / simplicidad** de todas las estrategias. Speedup cercano al ideal con una sola línea de directiva.

# Actividad 2: collapse(2)

```
#pragma omp parallel for collapse(2) \
        reduction(max:delta)
for (int i = 1; i < M; ++i)
    for (int j = 1; j < N; ++j) {
        // actualizacion V[i][j]
    }
```

### ¿Qué hace collapse(2)?

Fusiona los dos bucles en un único espacio de $(M-2) \times (N-2) \approx 10^6$ iteraciones, permitiendo distribuir cualquier subconjunto entre los hilos.

### ¿Por qué fue más lento?

- **Peor localidad de caché**: hilos distintos acceden a la misma fila $\rightarrow$ *false sharing*
- Mayor overhead de planificación sobre $\sim 10^6$ iteraciones

Útil solo cuando el bucle exterior tiene **pocas iteraciones**.

### Resultados

| Métrica | Valor |
|---|---|
| Tiempo | 138.58 s |
| Iteraciones | 236 161 |
| Speedup | 12,0× |

### Comparación

138.58 s vs 102.39 s de Act. 1
**+35 % más lento**.

Con 1022 filas para 20 hilos, `parallel for` ya tiene suficiente granularidad.

# Actividad 3: sections

```
// Serial primero: memoria debe existir
allocate_grid(M, N, V, h, k);

// Cuatro bordes en paralelo
#pragma omp parallel sections
{
    #pragma omp section
    { /* borde izquierdo: V(1,y)=(1-y)^2 */ }

    #pragma omp section
    { /* borde derecho:   V(2,y)=(2-y)^2 */ }

    #pragma omp section
    { /* borde inferior:  V(x,0)=x^2     */ }

    #pragma omp section
    { /* borde superior:  V(x,2)=(x-2)^2 */ }
}
```

## Resultados

| Métrica | Valor |
|---|---|
| Tiempo | 103.04 s |
| Iteraciones | 236 161 |
| Speedup | 16,1× |

### Decisión de diseño

`allocate_grid` debe ir **antes** (los bordes necesitan memoria ya reservada). Los 4 bordes son independientes entre sí → sin condición de carrera.

### ¿Vale la pena?

No. Inicialización es $\mathcal{O}(M+N)$, negligible frente a $\mathcal{O}(M \cdot N \cdot \text{iters})$ de `solve_poisson`.

# Actividad 4: parallel + for + schedule

```
bool converged = false;
#pragma omp parallel shared(V,delta,converged)
{
  while (!converged) {
    #pragma omp for reduction(max:delta) \
            schedule(static) // o dynamic
    for (int i = 1; i < M; ++i)
      for (int j = 1; j < N; ++j)
        { /* actualizar V[i][j] */ }

    #pragma omp single
    {
      ++iterations;
      if (delta <= TOL) converged = true;
      else              delta = 0.0;
    }
  } // while
} // parallel
```

## Resultados

| Schedule | Tiempo | Iter. |
|---|---|---|
| static | 110.46 s | 236 163 |
| dynamic | 129.43 s | 275 238 |

### static vs dynamic

**static**: bloques fijos, sin overhead. Óptimo con costo uniforme por fila.

**dynamic**: cola bajo demanda, más overhead. Rompe el orden Gauss-Seidel → más iteraciones.

### Bug crítico evitado

`break` dentro de región paralela viola el estándar OpenMP → hilos bloqueados indefinidamente en barrera. Solución: bandera `converged`.

### Ventaja de separar parallel del for

La región paralela se abre **una sola vez**, evitando overhead de crear/destruir hilos en cada iteración del `while`.

# Actividad 5: nowait, single, critical

```
// 1. single: imprime inicio de iteracion
#pragma omp single
{ cout << "Iter " << it << "\n"; }

// 2. Bucle principal (CON barrera)
#pragma omp for reduction(max:delta) \
        schedule(static)
for (int i = 1; i < M; ++i) { ... }

// 3. Bucle independiente (SIN barrera)
double local = 0.0;
#pragma omp for nowait schedule(static)
for (int i = 1; i < M; ++i)
  for (int j = 1; j < N; ++j)
    local += std::abs(V[i][j]);

// 4. Acumulacion protegida
#pragma omp critical
{ norma_L1 += local; }

// 5. Convergencia
#pragma omp single
{ if (delta<=TOL) converged=true;
  else { norma_L1=0.0; delta=0.0; } }
```

## Resultados

| Versión | Tiempo |
|---|---|
| nowait (sin print) | 115.02 s |
| nowait + print | 501.87 s |

**nowait: ¿cuándo usarlo?**

Solo cuando el código posterior **no depende** de los resultados del bucle actual. Aquí la norma L1 es independiente del cálculo de delta.

**critical + E/S = peligroso**

printf dentro de critical serializa toda la salida. Con ∼236 000 iteraciones y 20 hilos: 4,4× **más lento**. Nunca E/S en regiones críticas.

# Actividad 6: critical vs atomic

```
// Opcion 1: critical
// Protege cualquier bloque de codigo
// Mayor overhead: mutex de software
#pragma omp critical
{ ++iterations; }


// Opcion 2: atomic
// Solo para operaciones simples
// Instruccion atomica de hardware (LOCK XADD)
#pragma omp atomic
++iterations;
```

### Diferencia conceptual

**critical**: mutex de software, protege cualquier bloque. Overhead: adquisición/liberación de lock.

**atomic**: instrucción hardware (`LOCK XADD` en x86). Más liviana, solo operaciones simples.

### Resultados

| Mecanismo | Tiempo |
|---|---|
| critical | 105.86 s |
| atomic | 105.87 s |

### ¿Por qué no hubo diferencia?

El incremento ocurre **una sola vez** por iteración global del `while`, por lo que la contención es mínima con ambos.

Si estuviera dentro del bucle paralelo (una vez por nodo), la diferencia sería considerable.

### Regla general

Para `++contador`: prefiere siempre `atomic`. Menor overhead y mejor escalabilidad.

# Actividad 7: task + taskwait

```
#pragma omp parallel
{
  #pragma omp single
  {
    for (int bi=1; bi<M; bi+=BS)
      for (int bj=1; bj<N; bj+=BS)
        #pragma omp task
        {
          for (int i=bi;
               i<min(bi+BS,M); ++i)
            for (int j=bj;
                 j<min(bj+BS,N); ++j)
              /* actualizar V[i][j] */
        }
    #pragma omp taskwait
  }
}
```

## Resultados

| Métrica | Valor |
|---|---|
| Tiempo | 114.21 s |
| Iteraciones | 244 480 |
| Speedup | 14,5× |

### Ventajas de task

Paralelismo irregular y asimétrico. Ideal para algoritmos recursivos o dominios no estructurados donde `parallel for` no aplica.

### Desventaja en este caso

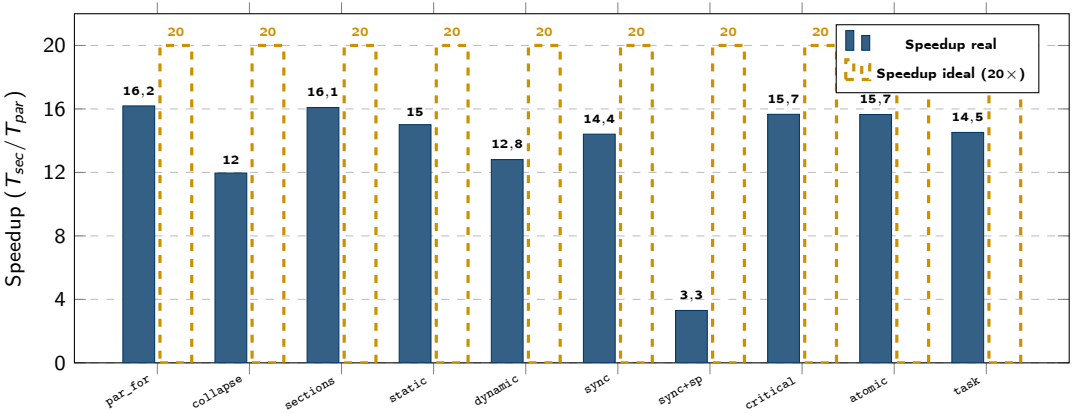
En patrones regulares, el overhead de crear/encolar/destruir tareas supera el beneficio. Las tareas se ejecutan en orden no determinista → rompe Gauss-Seidel → más iteraciones.

# Tabla Comparativa — Todas las Estrategias

| Versión | Directiva principal | Tiempo (s) | Iter. | Speedup |
|---|---|---|---|---|
| Secuencial | — | 1657.98 | 236 161 | 1,0× |
| **Paralelo básico** | `parallel for + reduction` | **102.39** | 236 164 | 16,2× |
| Collapse | `collapse(2)` | 138.58 | 236 161 | 12,0× |
| Sections | `sections` | 103.04 | 236 161 | 16,1× |
| Static | `parallel + for + static` | 110.46 | 236 163 | 15,0× |
| Dynamic | `parallel + for + dynamic` | 129.43 | 275 238 | 12,8× |
| Sync (sin print) | `nowait + single + critical` | 115.02 | 236 172 | 14,4× |
| Sync (con print) | `+ printf en critical` | 501.87 | 236 501 | 3,3× |
| Critical | `critical` | 105.86 | 236 166 | 15,7× |
| Atomic | `atomic` | 105.87 | 236 164 | 15,7× |
| Tasks | `task + taskwait` | 114.21 | 244 480 | 14,5× |

| Speedup ideal (20 hilos) | Speedup máximo obtenido | Peor caso |
|---|---|---|
| 20× | 16,2× (≈ 81 % eficiencia) | 3,3× (`critical`+E/S) |

# Speedup por Estrategia



# Directivas de Paralelización de Bucles

### `#pragma omp parallel for`

Crea una región paralela **y** distribuye las iteraciones del bucle en un solo paso. La región se abre y cierra en cada llamada.

**Cuándo usarla:** bucle independiente fuera de otro `while` o cuando la simplicidad es prioritaria.

**Overhead:** creación/destrucción de threads en cada invocación + barrera implícita al salir.

### `collapse(2)`

Fusiona $n$ bucles anidados en un único espacio de iteraciones de tamaño $N_1 \times N_2 \times \cdots \times N_n$.

**Cuándo usarla:** cuando el bucle exterior tiene **pocas iteraciones** (menos que el número de hilos) y el interior tiene muchas.

**Overhead:** mayor contención de caché y costo de planificación sobre el espacio fusionado. Puede producir *false sharing*.

### `#pragma omp parallel` + `#pragma omp for`

Separa la creación de la región paralela del reparto del bucle. Los threads se crean **una sola vez**; el `for` reparte iteraciones entre los ya activos.

**Cuándo usarla:** cuando dentro de la región paralela se necesitan otras directivas (`single`, `critical`, etc.) o cuando se itera múltiples veces sobre el mismo conjunto de threads (p.ej., dentro de un `while`).

**Overhead:** barrera implícita al salir del `for`. La creación de threads ocurre solo una vez.

### `reduction(op:var)`

Crea una copia privada de `var` por hilo, inicializada al elemento neutro del operador `op` (0 para `+`, $-\infty$ para `max`, etc.). Al salir del bucle, combina todas las copias con `op` y escribe el resultado en la variable compartida.

**Operadores:** `+`, `*`, `-`, `max`, `min`, `&`, `|`, `^`.

# Directivas de Planificación

### `schedule(static[, chunk])`

Divide las iteraciones en bloques de tamaño `chunk` y los asigna a los threads de forma **cíclica y determinista** antes de ejecutar.

**Cuándo usarla:** costo por iteración **uniforme** (todos los nodos tienen el mismo trabajo). Sin overhead de coordinación en tiempo de ejecución.

**Ejemplo:** 1022 filas, 20 hilos, sin chunk $\rightarrow$ cada hilo recibe $\approx 51$ filas fijas.

### `schedule(dynamic[, chunk])`

Mantiene una **cola de chunks** gestionada en tiempo de ejecución. Cada hilo solicita el siguiente chunk disponible al terminar el actual.

**Cuándo usarla:** costo por iteración **variable** (matrices triangulares, condiciones, geometrías irregulares).

**Overhead:** sincronización en cada solicitud de chunk + el orden de acceso a memoria cambia en cada iteración global.

### `schedule(guided[, chunk])`

Similar a `dynamic` pero los chunks comienzan grandes y se reducen exponencialmente hacia `chunk`. Buen balance entre overhead y adaptabilidad.

### Comparación de schedules

| | **static** | **dynamic** | **guided** |
|---|---|---|---|
| Overhead | Bajo | Alto | Medio |
| Balance carga | Fijo | Óptimo | Bueno |
| Determinista | Sí | No | No |
| Costo uniforme | Ideal | OK | OK |
| Costo variable | Pobre | Ideal | Bueno |

### Impacto en convergencia (este problema)

`dynamic` cambió el orden de actualización de celdas en cada iteración global $\rightarrow$ ruptura de Gauss-Seidel $\rightarrow$ **275 238 iteraciones** vs 236 163 de `static` (+16.5 %).

# Directivas de Sincronización

### `#pragma omp sections`

Divide el trabajo en bloques **cualitativamente distintos** (funciones diferentes) que se ejecutan en paralelo. Cada `section` la ejecuta exactamente un hilo.

**Cuándo usarla:** funciones independientes de costo similar que no comparten datos.

**Limitación:** el número de secciones limita el paralelismo (máx. = # secciones).

### `#pragma omp single`

Un **único hilo** ejecuta el bloque; los demás esperan en la **barrera implícita** al salir (a menos que se use `nowait`).

**Usos típicos:** inicialización de variables compartidas, impresión de diagnóstico, evaluación de criterios de convergencia.

### `#pragma omp nowait`

Elimina la barrera implícita al final de un `for`, `sections` o `single`. Los hilos que terminan primero avanzan sin esperar.

**Precondición:** el código posterior **no debe depender** de los resultados del bloque con `nowait`.

### `#pragma omp critical`

Solo un hilo a la vez puede ejecutar el bloque protegido. Usa un **mutex de software** (lock del SO).

**Cuándo usarla:** proteger bloques de código arbitrariamente complejos (estructuras, contenedores, E/S).

**Costo:** adquisición/liberación del lock, posible cambio de contexto. **Nunca** incluir E/S dentro de `critical` en bucles de muchas iteraciones.

### `#pragma omp atomic`

Garantiza que una **única operación simple** sobre una variable compartida se ejecute atómicamente usando instrucciones hardware (`LOCK XADD`, `LOCK CMPXCHG` en x86).

**Cuándo usarla:** `++x`, `x += val`, `x = min(x,y)`, etc. Siempre preferible a `critical` para operaciones simples.

**Costo:** mínimo — una instrucción hardware con prefijo `LOCK`.

### `#pragma omp task`

Crea una unidad de trabajo diferida que el runtime asigna dinámicamente a un hilo disponible. Ideal para paralelismo **irregular o recursivo**. `taskwait` sincroniza todas las tareas hijas.

# Comparativa de Métodos — Características Generales

| Característica | `par. for` | `collapse` | `sections` | `par+for` | `task` | `atomic` |
|---|---|---|---|---|---|---|
| Tipo de paralelismo | Datos | Datos | Funcional | Datos | Tareas | Variable |
| Threads creados por iteración | Sí | Sí | Sí | **No** | No | — |
| Overhead de sincronización | Bajo | Medio | Bajo | Bajo | Alto | Mínimo |
| Adecuado para bucles irregulares | No | No | — | Con dynamic | **Sí** | — |
| Adecuado para bucles regulares | **Sí** | Sí* | — | **Sí** | No | — |
| Preserva orden Gauss-Seidel | Parcial | No | — | Parcial | No | — |
| Permite otras directivas internas | No | No | Sí | **Sí** | Sí | — |
| Facilidad de uso | Alta | Alta | Media | Media | Baja | Alta |
| Escalabilidad | Alta | Media | Baja | Alta | Alta | Alta |
| Speedup obtenido (20 hilos) | 16,2× | 12,0× | 16,1× | 15,0× | 14,5× | 15,7× |

*** Solo cuando el bucle exterior tiene pocas iteraciones respecto al número de hilos.**

**Mejor para este problema**

`parallel for + reduction`
Mayor speedup, mínima complejidad.

**Mejor para bucles irregulares**

`task + taskwait`
Paralelismo dinámico y adaptativo.

**Evitar en este contexto**

`collapse(2)` y `dynamic`
Degradan rendimiento y convergencia.

# Comparativa de Métodos — Sincronización

| Característica | `single` | `nowait` | `critical` | `atomic` | `barrier` |
|---|---|---|---|---|---|
| Propósito | Un hilo ejecuta | Elimina barrera | Exclusión mutua | Op. atómica | Sincroniza todos |
| Implementación | Barrera implícita | Ninguna | Mutex SW | Instr. HW | Barrera SW |
| Costo por llamada | Medio | Nulo | Alto | Mínimo | Medio |
| ¿Serializa hilos? | Parcial (espera) | No | Sí | Mínimo | Sí (todos) |
| Operaciones protegibles | Cualquier bloque | — | Cualquier bloque | Solo simples | — |
| ¿Puede incluir E/S? | Sí (con costo) | — | Sí (muy costoso) | No | — |
| ¿Puede estar en bucle interno? | Con precaución | **Sí** | Evitar | **Sí** | Evitar |
| Uso típico en este taller | Convergencia | Norma L1 | Acumulación | Contador iter. | Implícita en `for` |
| Impacto en tiempo (este problema) | $+2{,}3\,\%$ | $-2{,}0\,\%$ | $+334\,\%$ (con E/S) | $\approx 0\,\%$ | Implícito |

## Regla de oro para sincronización

1. Usar `reduction` antes que `critical`/`atomic` cuando sea posible.
2. Si se necesita `atomic`: preferir sobre `critical` para operaciones simples.
3. Nunca colocar E/S dentro de `critical` en bucles de alto número de iteraciones.
4. `nowait` solo cuando se puede garantizar independencia de datos.

## ¿Cuántas barreras hay por iteración?

| Versión | Barreras/iter |
|---|---|
| `parallel for` | 1 (implícita) |
| `parallel+for` | 2 (for+single) |
| `parallel+for+nowait` | 2 (for+single) |
| `task+taskwait` | 1 (taskwait) |

Cada barrera implica espera de todos los hilos → reducirlas mejora el rendimiento.

# Conclusiones

## Lo que funcionó bien

1. `parallel for + reduction`: 16,2×, mejor relación rendimiento/simplicidad.
2. `sections`: correcto y seguro, beneficio marginal por el bajo costo de la inicialización.
3. `atomic` preferible a `critical` para operaciones simples: menor overhead hardware.
4. **Bandera** `converged`: única forma correcta de salir de un `while` en región paralela.

## Lo que degradó el rendimiento

1. `collapse(2)`: peor localidad de caché y *false sharing*.
2. `schedule(dynamic)`: overhead de cola y ruptura de Gauss-Seidel → más iteraciones.
3. `critical` + **E/S**: serialización total, 4,4× más lento.
4. `task` en patrones regulares: overhead de gestión de tareas supera el beneficio.

## Mensaje central

OpenMP permite aceleraciones cercanas al ideal (≈ 16× con 20 hilos) con pocas líneas de código, siempre que se eviten E/S dentro de regiones paralelas y se elija la estrategia de planificación adecuada al patrón de datos.

# ¿Preguntas?

Diego A. Acosta V. — dacosta@udistrital.edu.co
Oscar J. Cogua A. — ojcoguaa@udistrital.edu.co

Sistemas Distribuidos
Universidad Distrital Francisco José de Caldas

25 de marzo de 2026